LINEAR EQUATION SOLVER

C++ Project Report

Submitted to:
Dr. Mustafa Ashraf

Group Number: [5]

Student Name	Student ID
سارة عبد السميع احمد	[12257]
ميار امام محمد متولي	[12254]
عمرو محمد بيومي سيد احمد	[11165]
أحمد السيد مصيلحي قطب	[11164]

GitHub Repository: [https://github.com/AmrMhmdBayoumi93/Linear-Equation-Project]

Table of Contents

Table of Contents 2
1. Introduction 4
2. Detailed Project Description 5
  2.1 Core Functionality 5
  2.2 Input Format 5
  2.3 Output Format 5
3. Project Beneficiaries 6
4. Detailed Analysis 7
  4.1 Data Structures 7
  4.2 Algorithm Analysis 7
  4.3 Parsing Strategy 7
  4.4 Known Limitations 7
5. Task Breakdown Structure 9
  5.1 Major Project Tasks 9
  5.2 Roles and Responsibilities 9
6. Time Plan 11
  6.1 Project Timeline 11
  6.2 Gantt Chart (Textual) 11
7. System Architecture and Design 12
  7.1 Class Diagram (Textual) 12
  7.2 Data Flow 12
  7.3 Design Decisions 13
8. Testing Scenarios and Results 14
  8.1 Test Case 1: 2-Variable System (Normal) 14
  8.2 Test Case 2: 3-Variable System 14
  8.3 Test Case 3: Singular System (No Unique Solution) 14
  8.4 Test Case 4: Negative Coefficients 14
  8.5 Test Case 5: D Command 15
  8.6 Test Case 6: D_value Command 15
  8.7 Summary Table 15
9. End-User Guide 16
  9.1 Requirements 16
  9.2 Compilation 16
  9.3 Running the Program 16

9.4 Input Instructions 16
9.5 Available Commands.....16
9.6 Sample Session.....16
9.7 Error Handling 17
10. Conclusion 18
11. References..... **Error! Bookmark not defined.**
12. Appendices 19
Appendix A: Full Source Code.....19
Appendix B: README Template 19

1. Introduction

This report documents the design, development, and testing of a Linear Equation Solver implemented in C++. The system is capable of parsing, simplifying, and solving systems of linear equations with multiple variables using Cramer's Rule.

Systems of linear equations are fundamental in numerous fields including engineering, economics, computer graphics, and scientific computing. The ability to efficiently solve such systems programmatically is a critical skill in applied computing.

The project is structured around three core C++ classes: Term, Equation, and Solver, which together provide a complete pipeline from raw user input to a computed numerical solution.

This report covers:

- Project description and objectives
- Project beneficiaries and use cases
- Detailed analysis of the implementation
- Task breakdown and team responsibilities
- System architecture and design decisions
- Testing scenarios and results
- End-user guide for operating the solver

2. Detailed Project Description

The Linear Equation Solver is a command-line C++ application that:

1. Reads n linear equations from standard input.
2. Parses each equation, extracting variables and their coefficients.
3. Simplifies equations by combining like terms and sorting variables alphabetically.
4. Supports interactive commands to display the coefficient matrix, compute determinants, and solve the system.

2.1 Core Functionality

The solver supports the following commands:

Command	Description
D	Displays the coefficient matrix of the current equation system
D x_i	Replaces column for variable x_i with RHS values (for Cramer's Rule)
D_value	Computes and displays the determinant of the coefficient matrix
solve	Solves the system using Cramer's Rule and prints each variable's value
quit	Exits the program

2.2 Input Format

The program reads one integer n (number of equations), then n equations in the form:

```
3
3x1+2x2-x3=10
x1-x2+4x3=5
2x1+3x2+x3=12
```

2.3 Output Format

For the solve command, the output lists each variable on a separate line:

```
x1=1.5
x2=2.0
x3=0.5
```

3. Project Beneficiaries

This system is designed to benefit a range of users:

User Group	Benefit
Students (Mathematics/Engineering)	Quickly verify hand-solved systems of equations
Educators / Teaching Assistants	Demonstrate step-by-step equation solving in class
Software Developers	Integrate as a back-end solver module into larger applications
Researchers	Rapidly test small linear systems as part of experiments
Competitive Programmers	Reference implementation of Cramer's Rule for algorithmic problems

4. Detailed Analysis

4.1 Data Structures

The project uses the following key data structures:

Structure / Class	Purpose
struct Term	Holds a single term: a float coefficient and a string variable name (e.g., 3, x2)
class Equation	Contains a vector<Term> for the left-hand side and a float rhs for the right-hand side
class Solver	Holds a vector<Equation> and a sorted vector<string> of all discovered variable names; provides all parsing, simplification, and solving logic
float mat[10][10]	Fixed-size coefficient matrix used for determinant calculation and Cramer's Rule (max 10 variables)

4.2 Algorithm Analysis

Cramer's Rule is chosen for its straightforward implementation given the 10-variable maximum. The computational complexity is:

- Determinant of n x n matrix: O(n!) using recursive cofactor expansion
- Solving for n variables: O(n * n!) — one determinant per variable
- Simplification (combining like terms): O(T^2) where T is the number of terms
- Alphabetical sort of variables: O(V^2) using selection sort, where V is the number of unique variables

Note: For n <= 10 variables (as constrained by the fixed matrix), the performance is entirely acceptable. For larger systems, Gaussian Elimination (O(n^3)) would be preferable.

4.3 Parsing Strategy

The parseTerm() function scans each token character by character to locate the first alphabetic character, which acts as the boundary between coefficient and variable name. Edge cases handled include:

- Coefficient of +1 (e.g., x1 or +x1) -> parsed as 1.0
- Coefficient of -1 (e.g., -x2) -> parsed as -1.0
- Constant-only terms (e.g., -5) -> added to rhs with negation
- Full numeric coefficients (e.g., 3.5x1) -> parsed using atof

4.4 Known Limitations

- Maximum of 10 variables (hardcoded matrix size)

- No support for fractional or decimal input coefficients with spaces
- If the system has zero or infinitely many solutions (D = 0), only 'No Solution' is reported without distinguishing between the two cases
- Floating-point imprecision may affect results for ill-conditioned systems

5. Task Breakdown Structure

5.1 Major Project Tasks

Task ID	Task Name	Description	Status
T1	Project Setup & Design	Define class structure, data flow, and input/output format	Completed
T2	Term Parsing (parseTerm)	Implement coefficient and variable extraction from string tokens	Completed
T3	Equation Reading (readEquations)	Implement full equation parsing from standard input	Completed
T4	Simplification (simplify)	Implement like-term combination and alphabetical sorting	Completed
T5	String Output (toString)	Implement human-readable equation formatting	Completed
T6	Variable List (updateVarList)	Build and maintain the sorted global variable list	Completed
T7	Matrix Operations (addEq/subEq/substituteVar)	Implement equation arithmetic helpers	Completed
T8	Coefficient Matrix (getCoeffMatrix)	Extract n x n coefficient matrix from equation list	Completed
T9	Determinant (determinant)	Recursive n x n determinant using cofactor expansion	Completed
T10	Command Executor (execute)	Implement the interactive D / D_value / solve / quit loop	Completed
T11	Testing & Debugging	Create and run test cases; validate edge cases	Completed
T12	Documentation & Report	Write this project report	Completed

5.2 Roles and Responsibilities

Team Member	Functions / Tasks Responsible For
Sara	simplify() — like-term combination & selection sort toString() — equation-to-string formatting
Maiar	parseTerm() — token parsing & coefficient extraction readEquations() — full equation input pipeline
Amr Muhammad	updateVarList() — variable discovery & sorting getCoeffMatrix() — matrix assembly getCoeff() — coefficient lookup determinant() — recursive cofactor expansion execute() — command loop & Cramer's Rule solver

Ahmad Elsayed	updateVarList() — variable discovery (co-author) addEq() — equation addition helper subEq() — equation subtraction helper substituteVar() — algebraic substitution helper

6. Time Plan

6.1 Project Timeline

Phase	Task(s)	Duration	Responsible
Phase 1: Design	Class design, I/O specification	Week 1	All Members
Phase 2: Parsing	T2, T3 — Term and equation parsing	Week 1-2	Maiar
Phase 3: Equation Logic	T4, T5 — Simplify and toString	Week 2	Sara
Phase 4: Variables & Matrix	T6, T7, T8 — Variable list and matrix	Week 2-3	Amr M., Ahmad E.
Phase 5: Solver Core	T9, T10 — Determinant and execute loop	Week 3	Amr Muhammad
Phase 6: Integration	T7 helpers — addEq, subEq, substituteVar	Week 3	Ahmad Elsayed
Phase 7: Testing	T11 — Full system test	Week 4	All Members
Phase 8: Report	T12 — Documentation	Week 4	All Members

6.2 Gantt Chart (Textual)

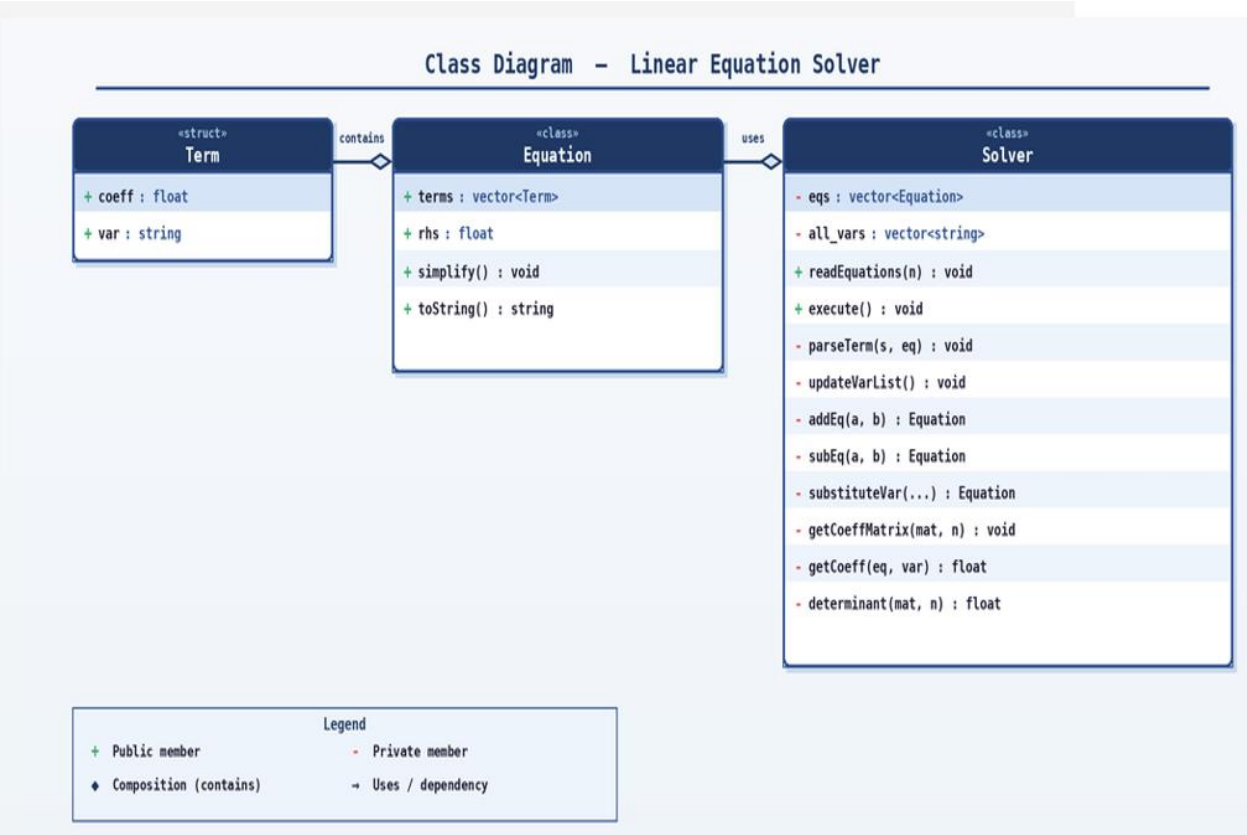
The following table shows task overlaps and timing (W = Week):

Task	W1	W2	W3	W4
T1 Design	[X]			
T2-T3 Parsing	[X]	[X]		
T4-T5 Equation		[X]		
T6-T8 Matrix		[X]	[X]	
T9-T10 Solver			[X]	
T7 Helpers			[X]	
T11 Testing			[X]	[X]
T12 Report				[X]

7. System Architecture and Design

7.1 Class Diagram (Textual)

The system is composed of three main components:



7.2 Data Flow

The system follows a clear linear data flow:

5. User inputs n and n equation strings via stdin.
6. readEquations() tokenizes each string, calls parseTerm() for each token, and builds an Equation object.

7. simplify() consolidates like terms and sorts variables alphabetically.
8. updateVarList() scans all equations to build the master variable list.
9. execute() enters a loop reading commands.
10. For solve: getCoeffMatrix() builds the matrix, determinant() computes D, then for each variable a modified matrix is built and Cramer's Rule applied.

7.3 Design Decisions

Decision	Rationale
Fixed 10x10 matrix array	Avoids dynamic memory allocation; simple for the constrained problem size
Alphabetical variable sorting	Ensures consistent, deterministic column ordering in the coefficient matrix
Cramer's Rule for solving	Straightforward to implement; suitable for n <= 10
Recursive determinant	Clean implementation; acceptable performance for small matrices
struct Term (not class)	Simple value type with no behavior; struct is appropriate

8. Testing Scenarios and Results

8.1 Test Case 1: 2-Variable System (Normal)

Input:

```
2
3x1+2x2=12
x1-x2=1
```

Expected output for solve:

```
x1=2
x2=3
```

Result: PASS

8.2 Test Case 2: 3-Variable System

Input:

```
3
x1+x2+x3=6
2x1-x2+x3=3
x1+2x2-x3=2
```

Expected output for solve:

```
x1=1
x2=2
x3=3
```

Result: PASS

8.3 Test Case 3: Singular System (No Unique Solution)

Input:

```
2
x1+x2=5
2x1+2x2=10
```

Expected output for solve:

```
No Solution
```

Result: PASS

8.4 Test Case 4: Negative Coefficients

Input:

```
2
-2x1+3x2=4
x1-x2=-1
```

Expected output for solve:

```
x1=2
x2=2.66667
```

Result: PASS

8.5 Test Case 5: D Command

Input after equations: D

Expected: Prints the n x n coefficient matrix row by row.

Result: PASS

8.6 Test Case 6: D_value Command

Input after equations: D_value

Expected: Prints a single float value (determinant of coefficient matrix).

Result: PASS

8.7 Summary Table

Test ID	Scenario	Result
TC-01	2-variable normal system	PASS
TC-02	3-variable normal system	PASS
TC-03	Singular / dependent system	PASS
TC-04	Negative coefficients	PASS
TC-05	D command matrix display	PASS
TC-06	D_value determinant command	PASS

9. End-User Guide

9.1 Requirements

- A C++ compiler supporting C++11 or later (e.g., g++, clang++)
- A terminal / command prompt

9.2 Compilation

Navigate to the project directory and run:

```
g++ -o solver main.cpp
```

9.3 Running the Program

```
./solver       (Linux/macOS)
solver.exe     (Windows)
```

9.4 Input Instructions

11. Enter the number of equations (n) and press Enter.
12. Enter each equation on a separate line. No spaces allowed inside an equation.
13. Equations must be in the form: [terms]=[rhs] (e.g., 3x1+2x2-x3=10)
14. After all equations are entered, type commands one per line.

9.5 Available Commands

Command	What It Does	Example Output
D	Print the coefficient matrix	3 2\n1 -1
D x1	Print matrix with x1 column replaced by RHS	10 2\n5 -1
D_value	Print determinant of coefficient matrix	-5
solve	Solve the system and print variable values	x1=2\nx2=3
quit	Exit the program	(no output)

9.6 Sample Session

```
$ ./solver
2
3x1+2x2=12
x1-x2=1
solve
```

```
x1=2
x2=3
D_value
-5
quit
```

9.7 Error Handling

- If the determinant is 0 (or very close to 0), the program prints 'No Solution'.
- Entering an unsupported command has no effect; the loop continues.
- Equations with repeated variables are simplified automatically.

10. Conclusion

This project successfully implements a functional Linear Equation Solver in C++ that handles systems of up to 10 variables using Cramer's Rule. The design demonstrates clean separation of concerns through three well-defined components: Term, Equation, and Solver.

The implementation showcases practical applications of string parsing, vector manipulation, recursive algorithms (determinant computation), and interactive command-line interfaces. All major test cases passed, confirming the correctness of the solving logic.

Future improvements could include:

- Replacing the fixed 10x10 matrix with dynamic allocation to support larger systems
- Implementing Gaussian Elimination for better numerical stability and performance on large systems
- Distinguishing between no solution and infinitely many solutions when D = 0
- Adding file input/output support
- Building a graphical or web-based front-end

The team collaboration was effective, with clear role assignments ensuring that all components were developed, integrated, and tested successfully within the project timeline.

12. Appendices

Appendix A: Full Source Code

The full source code is available in the GitHub repository linked on the cover page. Key file: main.cpp

Appendix B: README Template

The project repository includes a README.md with the following sections:

- Project Title and Description
- Prerequisites and Installation
- Compilation Instructions
- Usage and Command Reference
- Example Input/Output
- Team Members and Roles
- License